

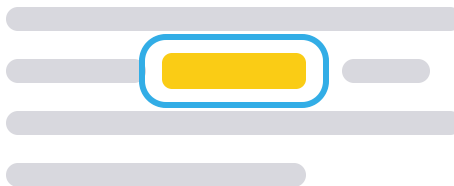
LookAsk 算法链路讲义

从摄像头到「指着哪个词」

普通摄像头眼动只有段落级精度，
这条算法链把它放大成词级可用的指向



眼睛：大概在看**这一段**



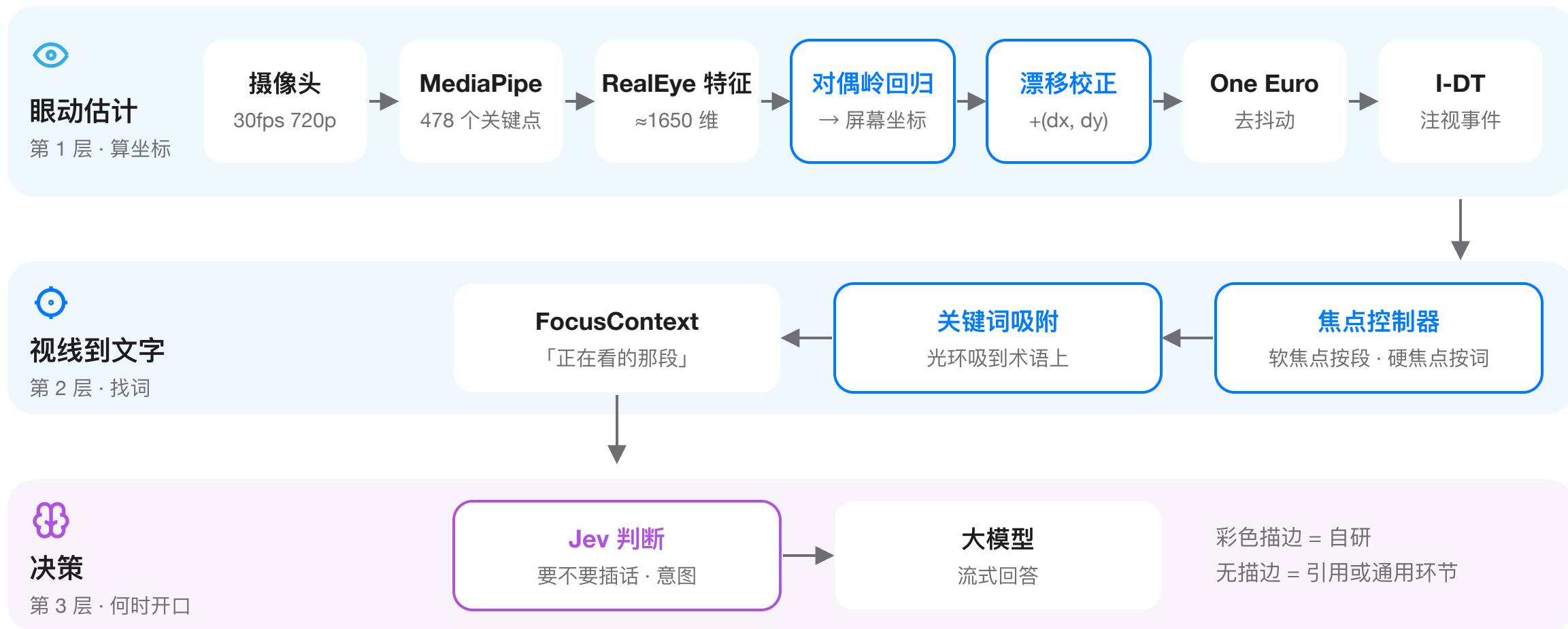
算法链 + 手柄：指到**这个词**



眼睛轮廓 = 摄像头看你在哪
虹膜 = 摇杆底座，琥珀点 = 焦点

一张图看懂：从摄像头到「指着哪个词」

整个系统是一条单向数据流，分成三层：眼动估计 → 视线到文字 → 决策



摄像头只能看清「哪一段」，所以每层都在补位

不跟物理极限硬拼，而是让每一层弥补上一层的不足

误差从哪来

-  眼睛在画面里只占几十个像素，虹膜的位移非常小
-  头一动，同样的眼球转角就对应不同的屏幕位置
-  光照、眨眼、眼镜反光都会带来噪声
-  校准之后人会慢慢挪动姿势，预测整体偏移（漂移）

结果：精度只到「段落级」

七层补位

	算法	弥补什么
● 估计	对偶岭回归	把特征映射成个性化屏幕坐标
● 校正	残差核回归	校准之后的漂移
● 平滑	One Euro 滤波	帧与帧之间的抖动
● 事件	I-DT 注视检测	切出「盯住了」的事件
● 语义	关键词吸附	落点到术语的最后一段距离
● 交互	两级焦点	眼动做不到的词级精度交给手
● 决策	Jev 级联判断	什么时候帮忙、怎么回答

第一步：把一张脸变成约 1650 个数字

这一层用现成的模型和特征库，是引用，不是自研



① 摄像头画面

每帧 1280×720
约 30 帧 / 秒
笔记本自带摄像头



② MediaPipe 人脸关键点

478 个 3D 关键点，其中
468 / 473 是右 / 左眼虹膜中心
52 个表情系数（如 eyeBlinkLeft）
头部姿态矩阵（平移 + 旋转）
GPU 推理 · VIDEO 模式



③ RealEye 组合特征

双眼裁成 40×20 灰度块
+ 关键点几何 + 表情 + 姿态

≈1650 维

每帧一个特征向量 x

眨眼帧直接丢掉

闭眼时眼部图像没有意义：blink > 0.45 跳过，
保持上一次视线；校准采集更严，要求 blink < 0.4

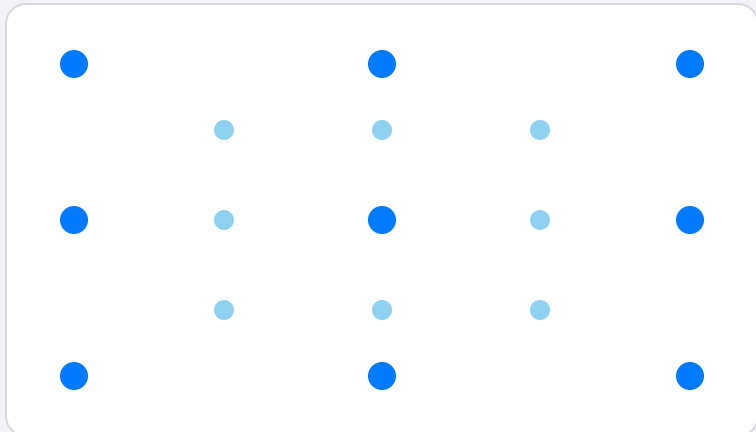
引用

这一层不是本项目的贡献

特征库 @realeye-io/webcam-eyetracker-
light-open 是 **AGPL-3.0**，分发时要遵守

校准：收集约 400 帧样本，训练一个带惩罚的线性映射

岭回归 = 最小二乘拟合 + L2 惩罚，防止权重被噪声撑得过大



9 或 17 个点，每点盯 850ms

每一帧记一条样本：

(特征向量 x ，目标点坐标 t)

共 $N \approx 400$ 帧，特征 $d \approx 1650$ 维

深蓝 = 9 点网格，浅青 = 17 点时加的内圈

目标：拟合误差 + $\lambda \times$ 权重大小

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|^2$$

预测：屏幕 x 、 y 各有一套权重 β

$$\hat{t}_x = \bar{t}_x + (x - \bar{x})^T \beta_x$$

λ 太小 $\rightarrow$ 过拟合

把噪声也记住了，
换个位置就不准

λ 太大 $\rightarrow$ 欠拟合

权重被压得太狠，
预测都挤向屏幕平均位置

特征比样本多，就换到样本空间里算

$d \approx 1650 \gg N \approx 400$ ：对偶形式把 1650×1650 的方程换成 400×400 ，快约十倍

原始形式·在特征空间算

$$\beta = (X^T X + \lambda I_d)^{-1} X^T y$$

要解一个 1650×1650 的方程组

构造 $X^T X$: $O(Nd^2)$

$\approx 1.1 \times 10^9$

次浮点运算

分解求解: $O(d^3/3)$

$\approx 1.5 \times 10^9$

结果
完全一样

对偶形式·在样本空间算

$$\alpha = (K + \lambda I_N)^{-1} y, \quad K = XX^T, \quad \beta = X^T \alpha$$

只要解一个 400×400 的方程组

构造核矩阵 XX^T : $O(N^2d)$

$\approx 2.6 \times 10^8$

次浮点运算

分解求解: $O(N^3/3)$

$\approx 2.1 \times 10^7$

怎么解：Cholesky 分解（手写，不依赖线代库）

$K + \lambda I$ 对称正定 $\rightarrow A = LL^T$ ，前代 $Lz = b$ ，回代 $L^T \alpha = z$

分解失败（浮点误差让矩阵「不够正定」）就把 $\lambda \times 10$ 重试

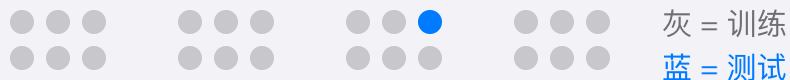
什么时候反过来？

样本比特征多 ($N \gg d$) 时，
原始形式反而更快

λ 怎么定：按校准点整组留出做交叉验证

自动选出正则强度，顺便得到一个不会偏乐观的精度报告

✗ 随机留一帧当测试



同一个点的其它帧几乎一模一样，还在训练集里 → 误差被低估

✓ 把一个校准点的所有帧整组留出



相当于预测一个从没见过的位置，误差才诚实

为什么不用神经网络

只有约 400 帧、却有 1650 维：小样本高维，容易过拟合
岭回归是闭式解，拟合 < 1 秒；每帧预测只是一次点积

完整流程

- 1 4 个候选 λ : $\{3e-4, 3e-3, 3e-2, 3e-1\} \times \text{diag}(K)$ 均值
- 2 每个 λ : 轮流留出一个点，用其余点训练
- 3 逐帧算欧氏误差再平均（贴近真实使用）
- 4 选误差最小的 λ ，用全部数据重新拟合
- 5 **cvErrorPx** 展示给用户；少于 5 个点不做 CV

像素误差 → 视角 ($1\text{pt} \approx 0.023\text{cm}$, 眼屏 55cm)

$$\theta = \arctan(\text{px} \times 0.023\text{cm} / 55\text{cm})$$

你每按一次确认键，都在悄悄帮它重新校准

用残差的时空加权平均修正漂移，收缩因子防止被一次误按带偏 · 自研

按 A / X / Y 确认

那一刻多半正看着焦点

记一条残差

真值 - 最近 450ms 预测均值

空间 × 时间加权

$\sigma = 520\text{pt} \cdot T = 20 \text{ 分钟}$

收缩因子 k

证据少时往 0 收

校正量 $\Delta(p)$

加到之后每一帧预测上

本质：Nadaraya-Watson 核回归（局部加权平均）

每条残差的权重 = 初始权重 × 离得近不近 × 新不新

$$w_i = w_i^0 \cdot \exp(-\|p_i - p\|^2 / 2\sigma^2) \cdot \exp(-(t - \tau_i) / T)$$

校正量 = 加权平均残差 × 收缩因子 k

$$\Delta(p) = k \cdot \sum w_i r_i / \sum w_i, \quad k = \sum w_i / (\sum w_i + 0.35)$$

屏幕不同区域可以有不同的偏移，而不只是一个全局平移

异常值剔除

真值和预测相差 $\geq 320\text{px}$ 直接丢掉
(差这么远，按键时多半没在看焦点)

权重分级

隐式校准权重 0.6；主动漂移校正权重 2
没有硬焦点时，盯屏幕中心点 1.5 秒

容量与衰减

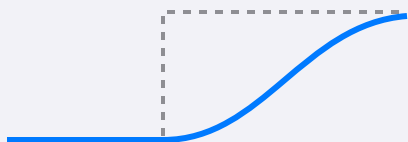
最多保留最近 16 条残差；
20 分钟前的残差权重只剩约 37%

盯住时要稳，扫视时要快：让截止频率跟着速度走

One Euro 滤波 (Casiez 2012) 同时解决抖动和延迟·引用

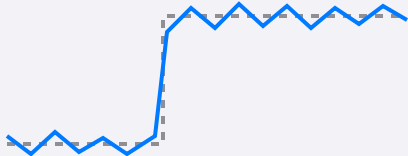
固定截止频率低

很稳，但眼睛跳走时
光环要慢半拍才跟上（延迟）



固定截止频率高

跟得快，但盯着不动时
光环一直在抖（抖动）



One Euro: 截止频率 = 下限 + $\beta \times$ 速度

$$f_c = f_{\min} + \beta \cdot |\text{速度}|$$

$$\alpha = 1 / (1 + \tau / \Delta t), \quad \tau = 1 / (2\pi f_c)$$

盯着不动 → 速度小 → f_c 低 → 强平滑，稳

扫视 → 速度大 → f_c 高 → 弱平滑，跟得上

项目里的参数

下限 $f_{\min} = 0.55$

$\beta = 0.0045$

速度先用 1Hz 低通滤一下，防止被噪声带偏

设置里的「平滑度」滑块 v : $f_{\min} = \max(0.12, 1.4 - 1.2v)$

x 、 y 各用一个独立的滤波器

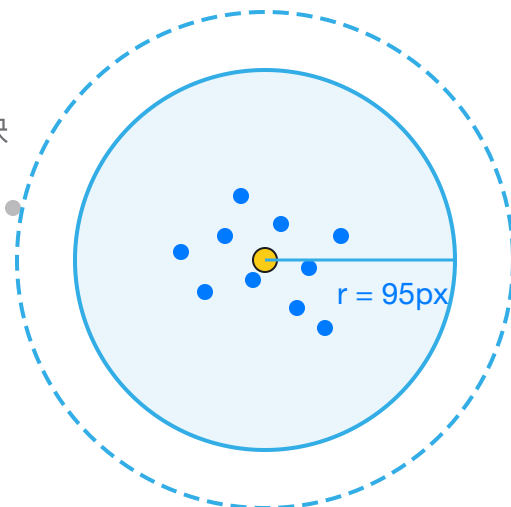
从视线流里切出「盯住了」：开始和结束用两个圈

读书时眼睛是「停一下 → 跳一下 → 再停」，只有停下来的注视才在读

上一次注视



扫视：点稀、跳得快



虚线外圈 $1.3 \times 95 \approx 124\text{px}$

琥珀点 = 质心，随新点不断更新

三条规则

开始 `fixStart`

最近 140ms 的点 (≥ 3 个)
全在质心 95px 内

持续 `fixUpdate`

新点离质心 $\leq 124\text{px}$
就并进来，更新质心

结束 `fixEnd`

新点跑出外圈就结束，
重新开始积累

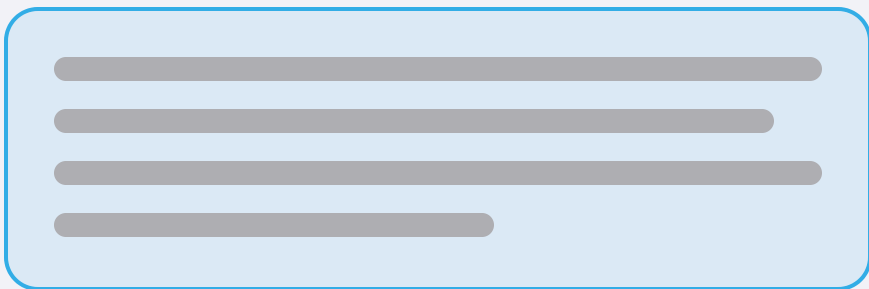
两个圈不一样大 = 迟滞：视线在边界抖动时不会反复开关；半径放大是为了扛住摄像头噪声

眼睛负责找到那一段，手负责点到那个词

两级焦点借鉴 MAGIC pointing (Zhai 1999)：视线粗定位，Joy-Con 摇杆精修·自研设计

👁️ 软焦点

视线（注视事件）驱动·按段落高亮·默认状态



第一次推右摇杆



看别处 > 0.8 秒
或按下右摇杆

🎮 硬焦点

右摇杆驱动·词 → 句 → 段 → 节，逐级切换



起步点：软焦点落点，或光环正吸着的那个词

左右分侧

手柄 - / + 切换看左边内容还是
右边回答；看另一侧时焦点不动，
贴边 70pt 内的注视拉回本侧

不轻易跳段 + 分词

视线出段不超过 stick 仍算本段；
词 / 句用 Intl.Segmenter 切分，
中文按词典分词

三种视图适配器

dom: Markdown / PDF / 对话气泡
terminal: xterm 终端
screen: 截屏 + OCR + 辅助功能取词

视线附近最像「值得问」的词，会把光环吸过去

关键词程度 × 距离衰减打分，迟滞防止来回跳，一个旋钮统一 9 个参数 · 自研

示例：视线落在这两行附近

我们用**对偶岭回归**求解，特征 $d \approx 1650$ ，



再用 One Euro 滤波去掉抖动。

青点 = 视线，灰点 = 探测网格 (caretRangeFromPoint 取词)

「对偶岭回归」是术语、离得近 → W 最高，被吸住

关键词程度 kw (启发式打分，封顶 1)

缩写 MDP、LLMs	0.9	中文 ≥ 4 字	0.8
字母 + 数字 GPT-4o	0.85	KaTeX 公式整块	0.95
驼峰 CamelCase	0.85	加粗 / 代码 / 标题里	+0.4
连字符 self-attention	0.72	中英文停用词	0

综合权重 (d = 视线到词框距离, r = 吸附半径)

$$W = kw \cdot \max(0, 1 - \min(1, d/r)^{1.5})$$

迟滞与扫视抑制 (防来回跳)

新词得分要高出 switchRatio 倍才换过去

视线离开超过 release 才松开

视线速度超过 saccade 时先不吸

🌀 单旋钮：强度 $k \in [0, 1]$ ，默认 0.7

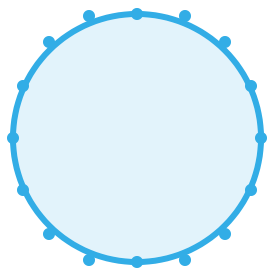
半径 $r = 40 + 100k$ 门槛 $= 0.65 - 0.5k$ stick $= 10 + 50k$

saccade $= 900(0.5 + 1.25k)$ release $= 110(0.5 + 1.25k)$

switchRatio $= 1.3 + (k - 0.4)$ 共 9 个参数线性联动

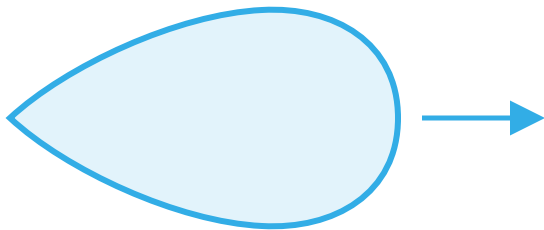
光环是一团果冻：会拉伸、会回弹、会包住词

质点—弹簧模型 + 超椭圆，让「吸住了哪个词」一眼看得见·自研



盯住：圆

36 个节点各自用弹簧
追静止位置，轻轻「呼吸」



扫视：泪滴

前沿弹簧硬、尾巴软，
尾巴最多拖出 42px



吸住：超椭圆

从圆渐变成包住词的
连续圆角框

$$|x/w|^5 + |y/h|^5 = 1$$

圆心弹簧 $k = 220$ 、阻尼比 0.92（几乎不过冲）；节点 $k = \max(45, 170(1 + 0.9\ell))$ ，阻尼比 0.5
 ℓ = 节点朝向与运动方向的一致度 · 每帧 2 次子步积分 · $n = 5$ 接近 iOS 连续圆角

先判断，再开口：便宜的判断放前面，贵的生成放后面

Jev 模式的三级级联，控制调用成本，也控制打扰·自研设计

① 本地规则（零成本）

段落太短不管·停留 < 1.5 秒不分析·同段 20 秒最多问一次



② Jev 判断（便宜，只做选择题）

难不难·卡住没·意图·要不要看图·要不要核实



③ 大模型（贵，真正写回答）

只有 Jev 判断需要时才调用

成本兜底（主进程）

内容哈希缓存最多 800 条·按天记账
超过每日 token 上限直接拒绝

例子：读者是不是卡住了？

- 1 停留 ≥ 9 秒或回看 ≥ 3 次，且难度 ≥ 1.4
→ 本地先判为「疑似卡住」
- 2 数字先分档成文字再交给 Jev（它读数字不可靠）：
停留 < 3 秒「很短」· 3-8 秒「正常」
8-20 秒「偏长」· > 20 秒「很长」
- 3 Jev 给出的概率 ≥ 0.65 ，才主动弹出解释

💬 指代消解：用语言补视线的精度

你说「这句什么意思」，视线只到段落 →
段落拆成候选句，Jev 结合原话挑出是哪一句
意图路由：解释 / 翻译 / 总结 / 推导 / 批判 / 其他
把握度低就不改写，按你的原话回答

哪些是拿来的，哪些是自己做的，去哪看

引用 4 项、自研 6 项，每一项都能在代码里找到

引用 现成模型、库和经典算法

MediaPipe 人脸关键点 (Google)

RealEye 组合特征库 (AGPL-3.0)

One Euro 滤波 (Casiez 等, 2012)

I-DT 注视检测 (Salvucci & Goldberg, 2000)

自研 自己实现或组合创新

对偶岭回归 + 分组交叉验证 (手写 Cholesky)

残差核漂移校正 + 隐式校准 + 异常值剔除

关键词吸附 + 迟滞 + 单旋钮参数化

两级焦点 (软 / 硬) + 左右分侧

质点-弹簧视线光环 + 超椭圆吸附

Jev 三级级联 + 指代消解

代码地图 (src/renderer/src/)

[gaze/engine.ts](#) 眼动主循环、漂移校正

[gaze/ridge.ts](#) 岭回归拟合与预测

[gaze/Calibration.tsx](#) 校准点、采集、ptToDeg

[gaze/filters.ts](#) One Euro、注视检测

[input/router.ts](#) 按键路由、隐式校准

[focus/focus.ts](#) 软 / 硬焦点控制器

[focus/magnet.ts, snap.ts](#) 关键词吸附、单旋钮

[gaze/blob.ts](#) 视线光环物理

[jev/jevBrain.ts](#) Jev 决策逻辑

完整推导与实验: docs/算法论文/LookAsk_paper.pdf

🔄 回来时先答这 7 个问题

每题先自己想，再看下面的答案要点

1

对偶形式为什么快？

$d \gg N$ 时在样本空间算，
解 400×400 而不是
 1650×1650 ； $N \gg d$ 时
反过来用原始形式

2

**交叉验证为什么
要按点整组留出？**

同一个点的帧几乎一样，
随机留帧会把误差
低估好几倍

3

收缩因子 k 干嘛用？

证据少或离得远时，
把校正量往 0 收，
防止一次误按带偏全屏

4

**One Euro 的 β
调大调小会怎样？**

调大：更跟手，但更抖
调小：更稳，但更拖

5

**I-DT 为什么开始和
结束用两个阈值？**

迟滞：视线在边界抖动时，
注视不会反复开始、结束

6

**switchRatio
起什么作用？**

也是迟滞：两个相近的词
之间，光环不会来回跳

7

**为什么数字要先
变成文字给 Jev？**

判断模型读数字不可靠，
分档成「很短 / 正常 /
偏长」更稳

**能答出这 7 问，
整条链路你就
真懂了**

答不上来就翻：
中文学习笔记 · 论文 §4-5